

CEO AI Thailand — คู่มือระบบ ฉบับทวนสอบ

Part 1 · ระบบ · กฎ · Skill · ความผิดพลาดและกลไกที่เหลือไว้

วันที่ทวนสอบ: 26 สิงหาคม 2569 — ทุกตัวเลขในเอกสารนี้ถูกนับใหม่จากรีโปและฐานข้อมูล production ในวันเดียวกัน ไม่ได้ลอกมาจากเอกสารเดิม เพราะบทเรียนข้อหนึ่งของโปรเจกต์นี้คือ เอกสารที่เคยถูกในอดีต จะกลายเป็นแหล่งความเชื่อผิดในอนาคต ถ้าไม่มีใครนับซ้ำ

วิธีอ่านป้ายในเอกสารนี้

ป้าย	ความหมาย	เงื่อนไข
●	ตรวจแล้ว	บอกได้ว่าตรวจยังไง · รันซ้ำได้
●	ยังไม่ได้ตรวจรอบนี้	เคยตรวจ หรือมาจากแหล่งที่เชื่อได้ แต่ไม่ได้นับใหม่วันนี้
●	ตรวจไม่ได้	พร้อมเหตุผลว่าเพราะอะไร และต้องทำอะไรถึงจะตรวจได้

● กฎสูงสุดของโปรเจกต์: ห้ามพูดสิ่งที่ยังไม่ได้ตรวจ
ตรวจไม่ได้ ให้บอกว่าตรวจไม่ได้ เพราะอะไร — ห้ามเดาแล้วเล่าเหมือนรู้
ความเร็วต้องยอมแพ้ความถูกต้องเสมอ — ตอบช้าแต่ถูก ดีกว่าตอบไวแล้วผู้ใช้เสียเวลาไปกับของที่ไม่จริง

1 · ระบบคืออะไร

CEO AI Thailand = AI Business Builder สำหรับคนไทย — พามาจาก "อยากมีธุรกิจ" → "มีลูกค้าจริง" → "เติบโตอย่างเป็นระบบ"

1.1 โครงสร้างแบรนด์

B.TC (บริษัทแม่ · Credibility Engine)
ตอบคำถาม "ทำไมเราถึงมีสิทธิ์สอนเรื่องการสร้างระบบธุรกิจ"
↓ powered by
CEO AI Thailand (ผลิตภัณฑ์ · Growth & Technology Engine)
ตอบคำถาม "AI จะทำให้ SME เริ่ม โต และ Scale ได้อย่างไร"
↓
START → GROW → SCALE → GOVERN
(Business (Marketing (Ops/KPI) (ISO/มอก./PDPA)
Fit/MIT24) OS)
← ขั้นสุดท้าย ไม่ใช่ปัญหาแรก

ทิศทาง: AI Growth Platform backed by B.TC Management-System Expertise

● ไม่ใช่: ISO SaaS ที่เพิ่ม Marketing เข้าไป

1.2 ขนาดระบบ (🟢 นับสดจากรีโป 26 ส.ค. 2569)

	จำนวน
บรรทัดโค้ดใน <code>src/</code>	88,372
โมดูลตรรกะ (<code>src/lib/*.ts</code>)	218
หน้า (<code>src/pages</code>)	45
คอมโพเนนต์	110
ไฟล์เทสต์	230
เทสต์ทั้งระบบ	2,624
migration	65
Edge Function	31
Skill	161

1.3 เทคโนโลยี

ชั้น	ของที่ใช้
Frontend	Vite + React + TypeScript (ไม่มี router — นำทางด้วย state)
Backend	Supabase (Auth · Postgres RLS · Edge Functions)
Production	Cloudflare Workers (<code>ceoaithailand.org</code>)
ฐานข้อมูลจริง	<code>waigsnxhrlwtiotspaim</code>
อีเมล	Resend (ส่ง) + Cloudflare Email Routing (รับ)
ตรวจสอบลิป	SlipOK (ตรวจกับ record ธนาคารจริง)

2 · กฎ — เฉพาะกฎที่ "กันได้จริง"

🔴 หลักการคัดของเข้าบั้น: กฎที่ไม่มีเทสต์บังคับ ไม่นับเป็นกฎ
มันคือความตั้งใจ และความตั้งใจจะถูกละเมิดในวันที่อดไม่เข้าเป้า

2.1 ลำดับชั้นของกฎ (บนทับล่างเสมอ)

① กฎสูงสุด — ความถูกต้องมาก่อน (ห้ามพูดสิ่งที่ยังไม่ได้ตรวจ)

↓

② รัฐธรรมนูญผู้ก่อตั้ง — Vision · Moonshot · Product DNA · Mission Chain

↓

③ Founder Mindset — 6 ด้านก่อนทำตามคำขอ (Validation ก่อน Scale)

↓

④ DMAIC — ทุกปัญหาต้องจบด้วย "ทางเลือกที่ดีที่สุด" ไม่ใช่รายการปัญหา

↓

⑤ Decision Rules – ลำดับคอขวด · observed ≠ inferred · ทุกเกณฑ์ติดป้าย

↓

⑥ Brand / Positioning / Moat – พูดยะไรได้ ที่ไหน กับใคร

↓

⑦ ด้านปล่อยของ – แต่ละ schema ได้หรือยัง

2.2 ตารางกฎ ↔ ไฟล์ ↔ เทสต์ (🟢 นับสดทั้งตาราง)

กฎ	ไฟล์	เทสต์	กันอะไร
รัฐธรรมนูญผู้ก่อตั้ง	founderConstitution.ts	constitutionReach	Golden Question — ตอบไม่ได้ = ไม่สร้าง
Founder Mindset	founderMindset.ts	28	ใช้เงิน/ขยาย ก่อนมีหลักฐาน
DMAIC	dmaic.ts	28	ส่งรายการปัญหาโดยไม่มีทางเลือก
Decision Rules	decisionRules.ts	18	อ่าน "นับได้" เป็น "อัตราจริง"
Brand Brief	brandBrief.ts	45	อ้างการยอมรับที่ยังไม่มี
โครงสร้างแบรนด์	brandArchitecture.ts	17	พาดหัวนำด้วย ISO · รวมสองแบรนด์
Positioning Engine	positioningEngine.ts	23	อ้างความต่างโดยไม่มี POD
แผนการแข่งขัน	competitiveStrategy.ts	15	ประกาศว่ามี moat ที่ยังไม่มี
Business Genome	businessGenome.ts	19	ฟีเจอร์ที่เพิ่มจำนวน ไม่เพิ่ม moat
จังหวะอารมณ์	emotionalArc.ts	22	ให้ความมั่นใจก่อนสร้างความตึง
ด้านปล่อยของ	releaseGates.ts	14	แต่ละ schema ก่อนผ่านด่าน
PDCA การเติบโต	growthPdca.ts	16	ตัดสินใจตอนเครื่องวัดยังตาบอด
เฟสของงาน	stageFit.ts	26	ทำงานปลายเกมตอนยังไม่มีลูกค้า
ตัวชี้วัดผลดำเนินงาน	opsMetrics.ts	19	ให้คะแนนก่อนมีข้อมูลพอ
Next Best Action	nextBestAction.ts	16	10 เอนจินตอบคนละทาง ไม่มีตัวบน
ทับ Skill ภายนอก	skillOverrides.ts	8	Skill ภายนอกสั่งสร้างแหล่งความจริงที่สาม

2.3 เทสต์ที่ "อ่านของจริง" ไม่ใช่ตรวจแต่โค้ด (🟢)

นี่คือหมวดที่มีค่าที่สุด เพราะเครื่องมือมาตรฐาน (`tsc` · `vitest` · `eslint` · `wrangler --dry-run`)
ตรวจได้แค่ว่าโค้ดถูก ไม่มีตัวไหนตรวจว่าโค้ดถูกเรียกใช้

เทสต์	อ่านอะไรของจริง
<code>workerRouting</code>	<code>wrangler.jsonc</code> + จำลอง matcher ของ Cloudflare
<code>ciGate</code>	<code>ci.yml</code> และ <code>cloudflare-deploy.yml</code> ตัวจริง
<code>rpcContract</code>	SQL ↔ TypeScript ตรงกันไหม
<code>envContract</code>	ตัวแปร env ถูกส่งเข้า build จริงไหม
<code>briefDocsContract</code>	ไฟล์ <code>.md</code> ตัวจริง (จับความผิดในเอกสาร)
<code>contentLinkContract</code>	คำสัญญาในโพสต์ ↔ เนื้อหาในบทความปลายทาง
<code>mobileFoldContract</code>	เพดาน padding + ลำดับ section + วิธีกันที่ของ fixed
<code>themeSafeColor</code>	ทั้ง CSS และตัวเครื่องมือตรวจเอง
<code>attributionContract</code>	รันสคริปต์ตัวจริงใน jsdom
<code>emotionalArc</code>	<code>LandingPage.tsx</code> ตัวจริง · เดินครบทั้ง 8 แบบที่ผู้ใช้เห็นได้
<code>releaseGates</code>	ทุกไฟล์ <code>.ts/.tsx/.md</code> ในรีโป — ห้ามมีไฟล์ไหนค้างสถานะเก่า
<code>ownershipBoundary</code>	รีโปนี้ห้ามมี <code>migration marketing_*</code>
<code>constitutionReach</code>	รัฐธรรมนูญไปถึง AI จุดไหนบ้างจริง

2.4 ด้านเดียวก่อน commit ทุกครั้ง

```
npm run ci = lint && build && test:run
```

- ⚠ ห้ามพูดว่า "เทสต์ผ่าน" ถ้ารันแค่ `vitest/typecheck` — `lint` คือขั้นที่ถูกลิ้มจน CI แดงติดกัน 54 รอบ 3 วัน
- ⚠ "push แล้ว" ≠ "อยู่บน production แล้ว" — ต้องเปิดดูผล GitHub Actions ด้วยตาเองก่อนบอกว่าเสร็จ

3 · Skill — 161 ตัว แต่ไม่เท่ากันทุกตัว

3.1 แยกที่มา (🟢) นับสดด้วย `comm` เทียบสองรายการ)

กลุ่ม	จำนวน	สถานะ
เขียนขึ้นในโปรเจกต์นี้เอง	60	เป็นกฎของเรา · หลายตัวมีเทสต์ผูก
sync มาจากภายนอก	101	เป็นความรู้ทั่วไป · ต้องผ่านด่านของเรา ก่อนใช้

3.2 Skill ที่เป็น "วิธีทำงาน" (บังคับใช้กับผู้ช่วย ไม่ใช่ความรู้)

Skill	ใช้เมื่อ
growth-mindset	ทุกครั้งก่อนส่งมอบงาน และทุกครั้งที่ถูกชี้ว่าผิด
shipped-not-written	ทุกครั้งก่อนพูดคำว่า "เสร็จ"
dmaic	ทุกครั้งที่เจอปัญหา
dynamic-plg	ทุกการแก้ไข/พัฒนา
moat-architecture	ก่อนออกแบบพีเจอาร์/ตาราง/agent ใหม่
beachhead-who-not-what	ก่อนเสนอให้ "โฟกัส/ตัดให้แคบลง"
case-study-stage-fit	ก่อนลอกกลยุทธ์จากเคสบริษัทใหญ่
experiment-reality-check	ก่อนเชื่อผลการทดลอง
invisible-influence	ก่อนสรุปว่าอะไรได้ผล/ไม่ได้ผล
content-link-contract	ก่อนปล่อยคอนเทนต์ทุกชิ้น
marketing-instruction	ก่อนลงมือทำการตลาด (คุม "ลำดับ")
theme-safe-color	ทุกครั้งที่แก้สีตัวอักษร
data-verification-10-rules	ก่อนนำตัวเลข/ข่าว/สถิติไปใช้

3.3 กฎการทับ Skill ภายนอก (🟢 skilloverrides.ts + 8 เทสต์)

● ของรีโปนี้ชนะเสมอเมื่อขัดกัน

ตัวอย่างจริงที่ต้องทับ:

- thai-marketing-strategy สั่งให้สร้าง BRAND.md แล้วเรียกมันว่า "แหล่งความจริงเดียว"
⇒ ทำตาม = ได้แหล่งความจริงเรื่องแบรนด์ ตัวที่สาม ซ้อนกับที่มีอยู่แล้ว 2 ที่
⇒ แหล่งความจริงคือ src/lib/brandBrief.ts เสมอ · ห้ามสร้าง BRAND.md ในรีโปนี้ (มีเทสต์เฝ้า)
- thai-performance-ads เขียนเกณฑ์ตัวเลขของตลาด ⇒ อ่านเป็น policy ห้ามเป็น validated

4 · ความผิดพลาด 55 ข้อ — จัดกลุ่มเป็น 8 รูปแบบ

🟢 นับสดจาก docs/LESSONS-LEDGER.md — 53 แถว · เลขล่าสุด #55

● หลักการของ ledger นี้: ผิดแล้วต้องเหลือ "กลไก" ไม่ใช่ "คำขอโทษ"

ทุกแถวบังคับให้มี 3 ช่อง: ผิดอะไร · สาเหตุราก · กลไกที่เหลือไว้

แถวที่ไม่มีกลไก = ยังไม่จบ

รูปแบบ A — รายงานว่าจริง/เสร็จ ก่อนไปดูของจริง

⚠️ รูปแบบนี้เกิดบ่อยที่สุด และทุกครั้งหลักฐานอยู่ในมือแล้ว

ตัวอย่าง	สิ่งที่เกิด
บอกว่า "ยังไม่ได้ยืนยัน Google Search Console" ทั้งวัน	ยืนยันไปแล้วตั้งแต่เดือนก่อน
รายงาน "เครื่องคำนวณโพล์พันขอบจอแล้ว"	ยังไม่โพล์
รายงาน "typecheck สะอาด"	<code>tsc --noEmit</code> ที่รากเป็น no-op — ไม่ได้ตรวจอะไรเลย
push โค้ดที่ compile ไม่ผ่านขึ้น main	—

กลไกที่เหลือไว้: `npm run ci` เป็นด้านเดียว · `ciGate.test.ts` อ่าน workflow ตัวจริง ·
กฎ "push แล้ว ≠ อยู่บน production แล้ว" · `run-ceo-ai-thailand` เปิดเบราว์เซอร์ดูจริงก่อนรายงาน

รูปแบบ B — เอา ตัวเลขที่วัดบั๊กของเราเอง ไปตั้งเป็นข้อเท็จจริงเรื่องตลาด

ตัวเลขที่เคยเชื่อ	ของจริง
<code>quickcheck = 0</code> ⇒ "ไม่มีใครสนใจ"	ช่องกรอกอยู่ใต้ขอบจอทุกเครื่อง
<code>search = 0</code> ⇒ "SEO ไม่ได้ผล"	เพิ่ง index ได้ 7 วัน
<code>social 32 / direct 45</code>	78 จาก 80 ลิงก์ไม่ติด utm
"คงผู้ชม 89% ⇒ คอนเทนต์ดีจริง"	คลิปเป็น Shorts ~20 วิ · 4.9 วินาทีวิว
"หยุดดูเฉลี่ย 38 วินาที"	timer เดินต่อหลังผู้ใช้สลับแอป

กลไกที่เหลือไว้: ก่อนตีความตัวเลขทุกครั้ง ถามก่อนว่า "เลขนี้วัดตลาด หรือวัดบั๊กของเรา"
และ ห้ามอ้างเปอร์เซ็นต์ใดโดยไม่ระบุตัวหาร

รูปแบบ C — เครื่องมือตรวจที่เดินไม่ครบ หรือให้คะแนนกลับด้าน

● อันตรรกะว่าไม่มีเครื่องมือ เพราะมันรายงานเขียว

เครื่องมือ	ความผิด
<code>contrast-audit</code>	เดินแค่ 6 หน้าจากเกือบ 30 แล้วรายงานเขียว
ตัววัดจังหวะหน้า Landing	regex <code>[a-z_]+</code> มองข้ามบล็อกที่มีตัวเลขในชื่อ (อ่าน 34 จาก 35)
ตัวนับความราบ	นับแบบ <code>floor(run/4)</code> ⇒ การเรียงที่แย่กว่าได้คะแนนดีกว่า
ตัวแก็สอัตรโนมิติ	เขียนสื้ที่ตัวมันเองบันทึกว่าไม่ผ่าน แล้วปล่อยขึ้นไป

กลไกที่เหลือไว้: ตัวตรวจทุกตัวต้อง พิมพ์จำนวนหน่วยที่เดินจริง และ ล้มเองถ้าเดินไม่ครบ ·
ตัวชี้วัดต้อง เรียงลำดับถูกทาง (มีเทสต์เทียบสองรูปแบบที่มีของเท่ากันเป๊ะ)

รูปแบบ D — เขียนถูก ≠ ถูกเรียกใช้

ตัวอย่าง	สิ่งที่เกิด
<code>run_worker_first</code> หายจาก <code>wrangler.jsonc</code>	Worker ไม่เคยถูกเรียกเลย ตอนคนเปิดหน้าเว็บ — ทั้งที่ได้ดเทสต์ และ dry-run ผ่านหมด
ประกาศว่า "Vision เดินทางไปกับทุก prompt"	ไปถึงจริง 1 จุดจาก 15
Amplitude	ไม่เคยได้ข้อมูลแม้แต่ event เดียว (env ไม่ได้ถูกส่งเข้า build)
<code>src/server.ts</code> + <code>src/agent/</code>	ไม่เคยถูกตรวจชนิดข้อมูลเลย (tsconfig exclude ไว้)

กลไกที่เหลือไว้: `workerRouting` · `envContract` · `constitutionReach` · `typecheck` สองฝั่ง

รูปแบบ E — แก้ในแชต ≠ แก้แล้ว

บทสนทนาหายไปกับ context window แต่เอกสารอยู่ต่อ — และรอบถัดไปเราจะอ่านเอกสารนั้นแล้วเชื่อมั่นอีก

- ยอมรับว่าพุดผิดเรื่อง GSC แล้ว แต่ไม่ได้ไล่แก้เอกสาร ⇒ ปฏิทินยังสั่งให้ไปทำงานที่เสร็จไปเดือนกว่าแล้ว
- สถานะ "ด้านปล่อยของ" ถูกเขียนกระจาย 14 ไฟล์ ⇒ วันที่มันเปลี่ยน ไม่มีอะไรบังคับให้ไล่แก้ครบ

กลไกที่เหลือไว้: `briefDocsContract` + `releaseGates` อ่านไฟล์ `.md` ตัวจริง ·

จบงานเมื่อ `grep` หาประโยคที่ผิดในทุกไฟล์แล้วแก้ครบ ไม่ใช่ตอนพิมพ์คำว่า "ผมผิด"

รูปแบบ F — แก้ทิศหนึ่ง แล้วเลยไปอีกทิศ

ตัวอย่างล่าสุด (26 ส.ค.): แก้ปัญหา "เวลาสูงเกินจริง" ด้วยการตัดจบตอนซ่อนแท็บ

⇒ คนที่สลับแอปแล้วกลับมาอ่านต่อ ไม่ถูกบันทึกเวลาอีกเลย = ต่ำเกินจริง

● ตัวเลขที่ต่ำเกินจริงตรวจยากกว่าตัวเลขที่สูงเกินจริง เพราะมันไปทางเดียวกับสมมติฐานที่เรามีอยู่แล้ว

กลไกที่เหลือไว้: แก่ที่ นิยาม ไม่ใช่ปิดสวิตช์ — นับ "เวลาที่อยู่ในสายตาจริง" ⇒ ถูกทั้งสองทิศ

รูปแบบ G — หมกมุ่นกับปัญหา แล้วส่งการตัดสินใจกลับไปให้เจ้าของ

เจ้าของทัก: "คุณหมกมุ่นแต่ปัญหา ๆ ต้องหาวิธีพัฒนาแล้วประเมินผล แล้วหาทางเลือกที่ดีที่สุดเลย · ผมอายุ 50 ปีแล้ว อย่าให้ผมเสียเวลาเยอะ"

รายการปัญหายาว ๆ ดูเหมือนความรอบคอบ แต่ถ้าไม่มีข้อเสนอ มันคือการผลักภาระ

— และผลักไปให้คนที่มีเวลาน้อยที่สุด

กลไกที่เหลือไว้: `dmaic.ts` — ด้าน Improve บังคับ ทางเลือก ≥ 3 ทาง รวม "ไม่ทำอะไรเลย" พร้อมคะแนน ·

บรรทัดแรกของคำตอบต้องเป็นข้อเสนอ ไม่ใช่ที่มา

รูปแบบ H — ความปลอดภัย / ข้อมูลรั่ว

ตัวอย่าง	สิ่งที่เกิด
ข้อมูลธุรกิจของเจ้าของถูกคัดลอกเข้าบัญชีผู้ใช้ใหม่ทั้งก้อน	รั่วข้ามบัญชี

ตัวอย่าง	สิ่งที่เกิด
รั้วข้ามบัญชีเกิด ซ้ำรอบสอง ทั้งที่แก้ไปแล้ว 23 ชั่วโมงก่อน	แก้ไม่ครบทาง
เกือบปล่อยตารางใหม่ให้ anon เข้าถึงได้	จับได้ก่อนขึ้น production

กลไกที่เหลือไว้: RLS ปิดเป็นค่าเริ่มต้น + เข้าผ่าน RPC ที่ guard ด้วย `is_app_admin()` · `rpcContract` เทียบ SQL ↔ TS

5 · สถานะจริงวันนี้ (🟢 query สดจาก production 26 ส.ค. 2569)

	จำนวน
workspace	3
รายการซื้อ skill ทั้งหมด	146
ที่จ่ายเงินจริง	0 (ที่เหลือเป็น admin-free)
แঙ্গโอน	0
หน้าร้านสาธารณะ	0
ผู้เข้าชม Landing (11–26 ส.ค.)	93
ในนั้นระบุแพลตฟอร์มที่มาได้	2 (2.2%)

🔴 ห้ามอ่านตัวเลขเหล่านี้ว่า "ตลาดไม่ตอบรับ" — ตามรูปแบบ B ข้างบน
ตัวเลขเหล่านี้ส่วนใหญ่วัด สภาพเครื่องมือของเรา ไม่ใช่ความต้องการของตลาด

5.1 อีกระบบหนึ่งเขียนลงฐานเดียวกัน (🟢 ยืนยันสด)

ฐาน production มีตาราง `marketing_*` 33 ตาราง ที่ไม่มีในรีโปนี้
⇒ รีโปนี้ ห้ามสร้าง migration `marketing_*` · ห้ามอ่าน/เขียนตาราง `marketing_*` (มีเทสต์บังคับ)

อีกระบบเป็นเจ้าของ	รีโปนี้เป็นเจ้าของ
การตลาดของเราเอง — แคมเปญ · คอนเทนต์ · brand rules · segment	ตัวผลิตภัณฑ์ — แอปที่ผู้ใช้ล็อกอิน · Landing · Worker · billing
Marketing OS · Instruction Profiles	รัฐธรรมนูญ · Founder Mindset · DMAIC · Business Genome · VRIO/Moat

6 · จุดบอดที่ยังเหลือ — พุดให้ตรง

#	จุดบอด	ทำอะไรถึงจะหาย
1	● utm_source = 2 จาก 93 ⇒ ตอบไม่ได้ว่าแพลตฟอร์มไหนได้ผล	ติดแท็กตอนปล่อยลิงก์ — แก่ด้วยโค้ดไม่ได้
2	● เกณฑ์ผ่านของ Phase 1 Acceptance #2 ไม่มีเขียนไว้ที่ไหนในรีโป	เจ้าของบอกเกณฑ์ แล้วผมทำเป็นเทสต์
3	● รัฐธรรมนูญยังไม่ถึง AI ทุกจุด (1 จาก 15 จุดเดิม · แก่ไปบางส่วนแล้ว)	ต่อสายให้ครบทุกจุดเรียก
4	● หน้า Landing สูง 52 จอ และ 86% ไม่เลื่อนเลย	ของที่อยู่ใต้ขอบจอ = แทบไม่มีใครเห็น
5	● ยังไม่มีลูกค้าจ่ายเงินจริง ⇒ ห้ามอ้างรีวิว/จำนวนผู้ใช้/เคสความสำเร็จ	ต้องมีของจริงก่อน

7 · Part 2 จะตอบอะไร

เจ้าของตั้งโจทย์ไว้: "คนที่เข้ามาในระบบต้องเกิด aha! แล้วเกิด viral"

Part 1 (เอกสารนี้) ตอบว่า ระบบคืออะไร และกฎอะไรกันเราไม่ให้โกหกตัวเอง

Part 2 จะตอบว่า aha! ของเราอยู่ตรงไหน วัดยังไง และอะไรทำให้มันถูกส่งต่อ

โจทย์ที่ Part 2 ต้องแก้ ตั้งอยู่บนข้อเท็จจริงจาก Part 1:

1. aha! ที่มีอยู่แล้วและพิสูจน์ได้ = ผู้ใช้เห็น ตัวเลขของตัวเอง เป็นครั้งแรก

(เครื่องตรวจสินค้าเร็ว · เครื่องประเมินตลาด · เครื่องคำนวณราคา) — ฟรี ไม่ต้องสมัคร แต่งตัวเลขไม่ได้

1. ● แต่ aha! ที่ไม่มีใครเห็น = ไม่มี aha! — 86% ไม่เลื่อนหน้าเลย

⇒ งานของ Part 2 ไม่ใช่ "สร้าง aha! ใหม่" แต่คือ ย้าย aha! ที่มีอยู่ขึ้นมาให้ถึงตา

1. viral ต้องมีของให้ส่งต่อ — และของที่คนส่งต่อได้ ต้องเป็น ตัวเลขของเขา ไม่ใช่โฆษณาของเรา

2. ● และต้องวัดได้ว่ามันเกิดจริง — ซึ่งวันนี้ยังวัดไม่ได้ เพราะ utm ครอบคลุม 2.2%

เอกสารนี้จบ Part 1

ทุกตัวเลขในเอกสารนี้รันซ้ำได้ · ถ้าตัวเลขไหนไม่ตรง แปลว่าเอกสารเก่าไปแล้ว ให้เชื่อการนับใหม่เสมอ